

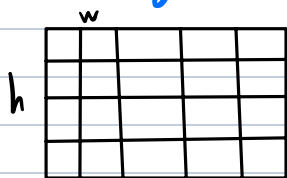
Chapter 0

How Neural Networks "See" Images?

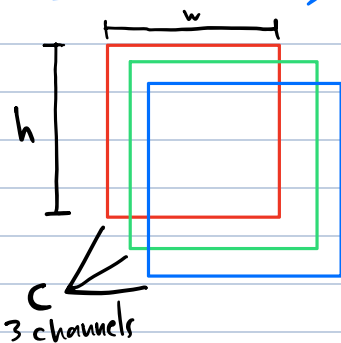
1. How images are represented as tensors?

An image is made up of tiny bright dots called **Pixels** with value (0-255), there are two ways to represent them for:

- **Grayscale**: using **2D tensors** (height x width) (0-255 / 0-1) ^{black white normalized}



- **Color Images (RGB)**: using **3D tensors** (height x width x channels) each channel has separated value (0-255/0-1)



⊗ In Standard DL Frameworks (**PyTorch**) the tensor is usually:

B x **C** x **H** x **W**

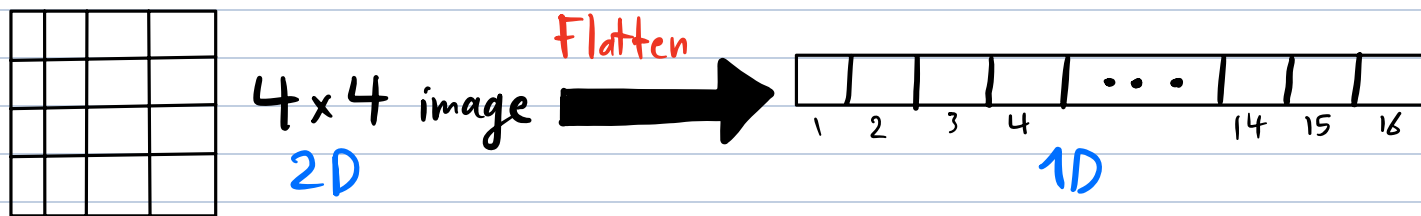
(Batch size x channels x height x width)

example

(5, 3, 256, 256) → 5 images with 3 channels
& size of 256 x 256

2. Why CNN replaced Fully connected Neural Networks (MLP)?

Early attempts of Computer Vision used MLP. In order to do so, a 2D image must be Flattened into a 1D vector.



This approach fails for 3 main reasons:

① **Destruction of Spatial Hierarchy**: Flattening a 2D image destroys the 2D spatial relationships between pixels. The network loses the context that pixel at index i is physically above the pixel at $i+5$.

② **Curse of Dimensionality**: An MLP needs $3 \times 256 \times 256$ weights for a neuron in the first hidden layer.

③ **Lack of translation Invariance**: If a cat is at the top-left part of the image, MLP won't be able to recognize it if it was at the bottom-right

3. Convolutions & Kernels

Instead of looking at the whole image at once, CNN looks at small, localized patches using Kernels/Filters (matrix of learnable weights e.g. 3×3).

During a convolution this Kernel slides across the input image. At every step it performs an element-wise multiplication with aligned pixels & sums it into a

single value (this value represents a feature based on the layer)

← ممكن تكون مجرد curve أو edge وإذا layers بعيدة ممكن وجه البسة... الخ

4. Feature Maps

the output matrix outputed by a convolution

→ if you apply a vertical edge detection filter the resulting feature map will have the vertical edges lightened up

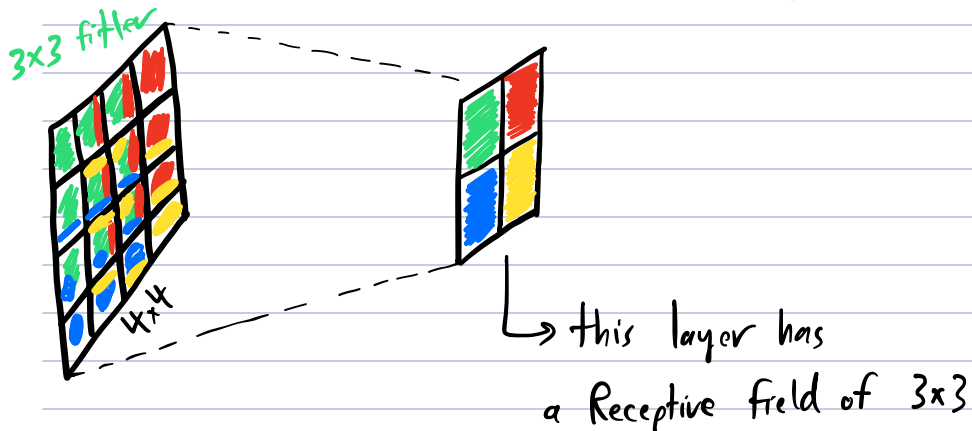
⊗ A single conv layer applies multiple different Kernels (e.g 64) resulting in a tensor with 64 different feature maps
↳ 64 different features

5. Receptive Fields

Refers to the specific region of the original input image that influences a particular value in a feature map

As we go deeper and deeper in the network convolutions are applied to the feature maps of previous convs.

Consequently, a single neuron in a deep layer might have a receptive field that encompasses the entire original image allowing it to see the global context



6. Stride & Padding

- Stride: Step size the Kernel takes as it slides. 1, 2, ...

Change the size of the resulting Feature map (downsampling the spatial dimension)

- Padding: Standard conv shrinks the output feature map's size & under-sample the edges (network doesn't see them that much). Padding adds fake (zeros) around the image border to process edge pixels fully & preserve the original spatial dimension

Size of feature map

$$O = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

7. Pooling

Downsampling the spatial dimensions of feature maps (height & width) while leaving the depth (channels) unchanged. Most common is Max Pooling

Purpose 1: Dramatically reduces computational load & memory requirements.

Purpose 2: Provides translation invariance. By only keeping the strongest activation in a location region, the network cares less about the exact pixels location & more that the feature exists somewhere in the area.

8. Hierarchical Feature Extraction (Deeper = More Complex)

Because of Pooling & stacking conv layers, CNNs learn a hierarchy of features

Shallow layers	Mid-level layers	Deep layers
Receptive fields are smaller	Combining lines & edges thes kernels	Receptive fields cover the whole image
Kernels detect: lines, edges & color blobs	detect: geometric shapes → wheels & eyes	by combining shapes Kernels can detect complex objects → cars, faces, dog breeds ...

9. Backbone (From Pixels to Semantics)

is the sequence of conv & pooling layers of the neural network

e.g ResNet, VGG, EfficientNet...

As the image undergoes, a critical transformations happen:

Spatial compression: H & W shrink ($256 \times 256 \rightarrow 6 \times 6$)

Channel expansion: (From 3 RGB Channels to 512, 1024 or more feature maps)

The backbone transforms a high-res image of meaningless pixels ($3 \times 256 \times 256$) into low-res, deeply semantic vector representation ($2048 \times 8 \times 8$)

At the very end of the network, this vector is usually flattened & passed into a final classifier head to make predictions

10. Transfer Learning VS. Fine-Tuning

The usage of Pre-trained models/backbones

⊗ Transfer-learning: take a pre-trained backbone, freeze all of its weights so they cannot be updated and only swap out the classifier head

We are transferring the model's ability to extract features & only train a new head to map those features to our specific classes

?? When to use ??

very small dataset & similar to the data that the model originally was trained on

⊗ Fine-tuning : Initialize the model with pre-trained weights but unfreeze some (deeper ones) OR all of the backbone layers. During training, the gradients update the feature extractors to better suit our new data.

?? When to use ??

Large dataset OR Your domain is significantly different from the pre-training domain (medical is a common example)